

Name-Moksh Jain | USN-24BTRCL126 | NLP PROJECT | CSE AIML(C)



Customer Feedback Analyzer using Sentiment Analysis

Objective: Build a simple NLP model that classifies customer feedback into **Positive**, **Negative**, or **Neutral** sentiments.

This notebook is designed for **beginners** and covers:

- Data loading & exploration
- Text preprocessing (lowercase, punctuation removal, stopwords, tokenization)
- TF-IDF vectorization
- Model training (Logistic Regression & Naive Bayes)
- Evaluation (accuracy, confusion matrix, classification report)
- Custom prediction
- Visualizations (sentiment distribution, confusion matrix, word cloud)

Step 1: Import Necessary Libraries

We start by importing all the libraries we'll need. These include:

- **pandas / numpy** for data manipulation
- **matplotlib / seaborn** for visualization
- **sklearn** for machine learning
- **nlTK** for natural language processing

```
# — Core Libraries —
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import string
import warnings
warnings.filterwarnings('ignore')

# — NLP Libraries —
import nltk
nltk.download('punkt', quiet=True)
```

```

nltk.download('punkt_tab', quiet=True)
nltk.download('stopwords', quiet=True)
nltk.download('wordnet', quiet=True)
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

# — Machine Learning Libraries —
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)

print("✅ All libraries imported successfully!")

```

✅ All libraries imported successfully!

✓ Step 2: Load / Create a Sample Dataset

Since we want this notebook to run without any external files, we'll create a **sample dataset** directly in code.

The dataset has two columns:

- **review** – the customer feedback text
- **sentiment** – the label (Positive / Negative / Neutral)

```

# — Sample Customer Feedback Dataset —
data = {
    'review': [
        # — Positive Reviews —
        'This product is amazing and works perfectly!',
        'Absolutely love it, best purchase I have ever made.',
        'Great quality and fast delivery. Highly recommend!',
        'Excellent customer service and wonderful product.',
        'Very happy with my purchase, will buy again.',
        'The quality exceeded my expectations, truly fantastic.',
        'Outstanding performance and beautiful design.',
        'I am so impressed with this item, it is perfect.',
        'Five stars! Everything was smooth and easy.',
        'Superb quality, worth every penny spent.',
        'Wonderful experience from start to finish.',
        'This is exactly what I was looking for, love it!',
    ]
}

```

```
'Top notch product with premium quality materials.',
'Brilliant product, my whole family loves it.',
'Fast shipping and the item arrived in perfect condition.',
'I am extremely satisfied with this purchase.',
'The best product in this price range, hands down.',
'Incredibly well made, you can feel the quality.',
'A joy to use every single day, highly recommended.',
'Customer support was friendly and very helpful.',

# — Negative Reviews —
'Terrible product, broke after one day.',
'Worst purchase ever, total waste of money.',
'Very disappointed with the quality, feels cheap.',
'The product stopped working within a week.',
'Awful customer service, no one responded to my complaint.',
'Not as described, completely misleading advertisement.',
'Horrible experience, I want a full refund.',
'The packaging was damaged and the item was broken.',
'Do not buy this, it is a complete scam.',
'Extremely poor quality, falling apart already.',
'I regret buying this, nothing works as promised.',
'Shipping took forever and the product was defective.',
'This is the worst product I have ever used.',
'Cheap materials and terrible craftsmanship.',
'Returned it immediately, totally unacceptable quality.',
'Waste of money, does not work at all.',
'The product looks nothing like the pictures online.',
'Received a damaged item and customer care was unhelpful.',
'Very frustrating experience, would give zero stars if I could.',
'Overpriced garbage, avoid at all costs.',

# — Neutral Reviews —
'The product is okay, nothing special.',
'It works fine but could be better.',
'Average quality for the price.',
'Not bad, but not great either.',
'Decent product, does what it is supposed to do.',
'The product met basic expectations, nothing more.',
'It is an alright product for everyday use.',
'Fairly standard, no complaints but no excitement.',
'The product is functional but lacks premium feel.',
'Mediocre quality, expected a bit more for the price.',
'It does the job, nothing exceptional though.',
'Packaging was fine, product is just average.',
'Neither impressed nor disappointed, just okay.',
'Acceptable quality, might consider other options next time.',
'Somewhat useful, but I have seen better alternatives.',
'It arrived on time, product is passable.',
'Standard product, serves its purpose adequately.',
'Regular quality, nothing to write home about.',
'The product is satisfactory but unremarkable.'
```

```

        'Mixed feelings about this one, it is just so so.'
    ],
    'sentiment': (
        ['Positive'] * 20 +
        ['Negative'] * 20 +
        ['Neutral'] * 20
    )
}

df = pd.DataFrame(data)
print(f"Dataset shape: {df.shape}")
print(f"Sentiment distribution:\n{df['sentiment'].value_counts()}")
df.head(10)

```

```

Dataset shape: (60, 2)
Sentiment distribution:
sentiment
Positive    20
Negative    20
Neutral     20
Name: count, dtype: int64

```

	review	sentiment
0	This product is amazing and works perfectly!	Positive
1	Absolutely love it, best purchase I have ever ...	Positive
2	Great quality and fast delivery. Highly recomm...	Positive
3	Excellent customer service and wonderful product.	Positive
4	Very happy with my purchase, will buy again.	Positive
5	The quality exceeded my expectations, truly fa...	Positive
6	Outstanding performance and beautiful design.	Positive
7	I am so impressed with this item, it is perfect.	Positive
8	Five stars! Everything was smooth and easy.	Positive
9	Superb quality, worth every penny spent.	Positive

Next steps: [Generate code with df](#) [New interactive sheet](#)

✓ Step 3: Exploratory Data Analysis (EDA)

Let's take a quick look at how our sentiments are distributed in the dataset.
A balanced dataset helps the model learn each category equally well.

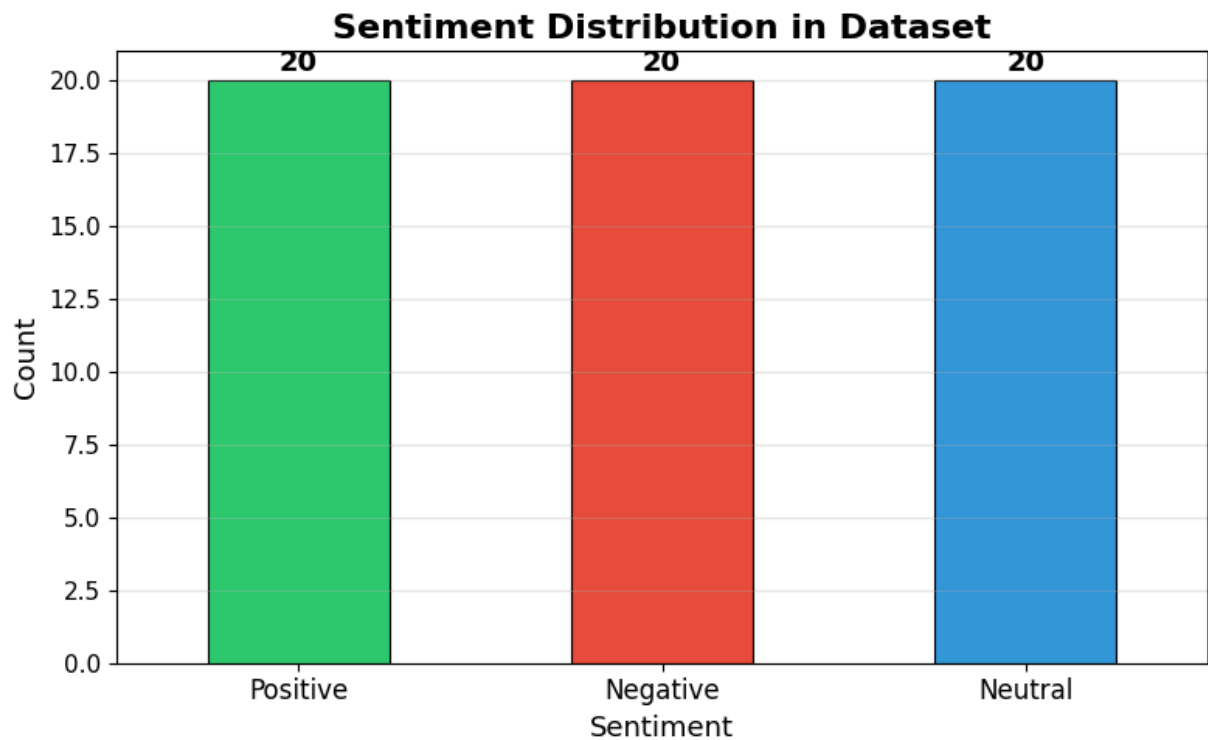
```
# — Sentiment Distribution Bar Chart —
plt.figure(figsize=(8, 5))

colors = ['#2ecc71', '#e74c3c', '#3498db'] # Green, Red, Blue
sentiment_counts = df['sentiment'].value_counts()

ax = sentiment_counts.plot(
    kind='bar',
    color=colors,
    edgecolor='black',
    linewidth=0.8
)

# Add value labels on top of each bar
for i, (val, name) in enumerate(zip(sentiment_counts.values, sentiment_counts.index)):
    ax.text(i, val + 0.3, str(val), ha='center', fontweight='bold', fontsize=13)

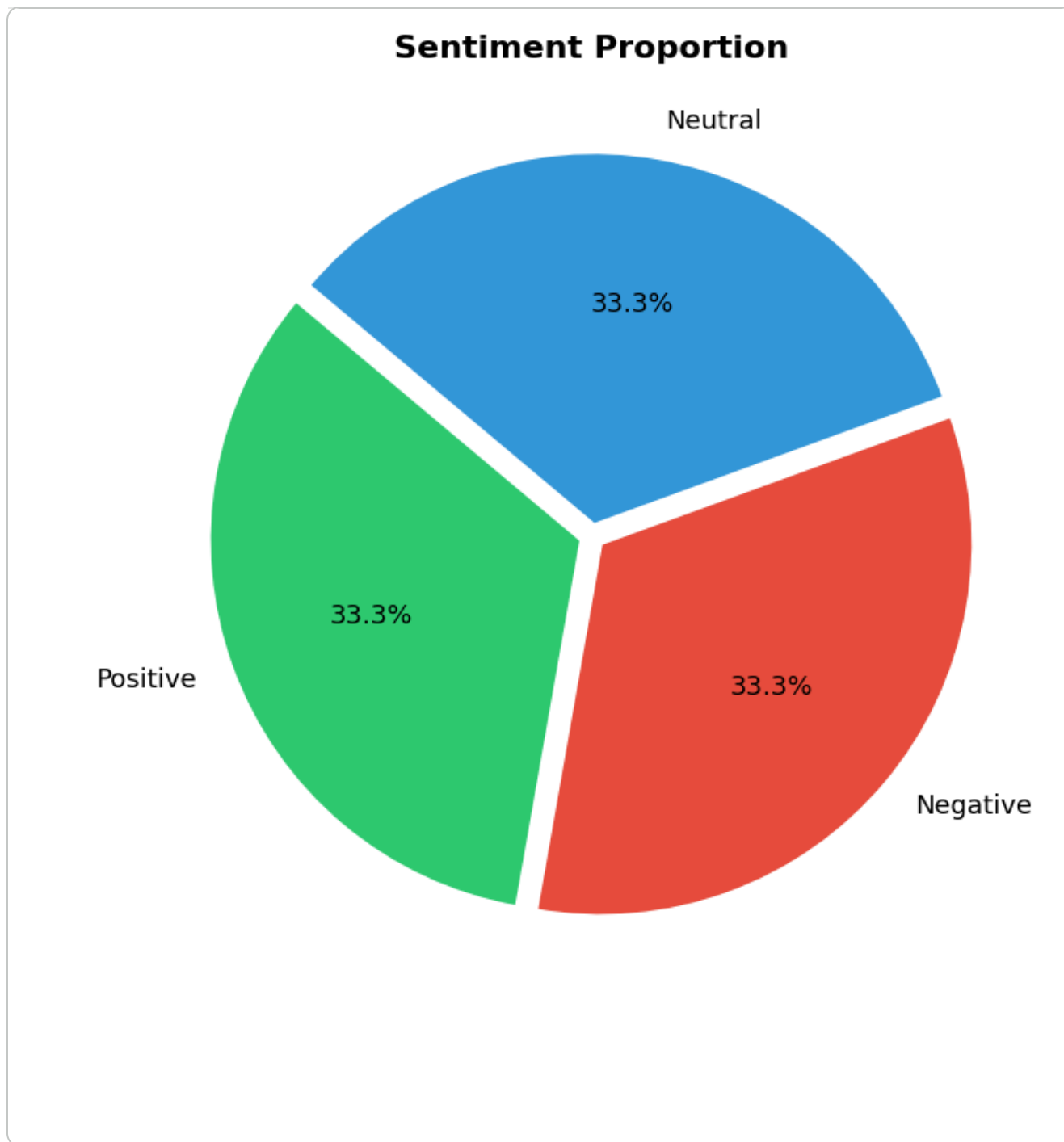
plt.title('Sentiment Distribution in Dataset', fontsize=16, fontweight='bold')
plt.xlabel('Sentiment', fontsize=13)
plt.ylabel('Count', fontsize=13)
plt.xticks(rotation=0, fontsize=12)
plt.yticks(fontsize=11)
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```



```
# — Sentiment Distribution Pie Chart —
plt.figure(figsize=(7, 7))

plt.pie(
    sentiment_counts.values,
    labels=sentiment_counts.index,
    colors=colors,
    autopct='%1.1f%%',
    startangle=140,
    textprops={'fontsize': 13},
    wedgeprops={'edgecolor': 'white', 'linewidth': 2},
    explode=(0.03, 0.03, 0.03)
)

plt.title('Sentiment Proportion', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()
```



✓ Step 4: Data Preprocessing

Before feeding text data to a machine learning model, we need to clean it. Our preprocessing pipeline will:

1. Convert text to **lowercase**
2. Remove **punctuation**
3. **Tokenize** the text (split into individual words)
4. Remove **stopwords** (common words like 'the', 'is', 'and')
5. Apply **lemmatization** (reduce words to their base form)

```

# — Initialize tools —
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def preprocess_text(text):
    """Clean and preprocess a single text string."""
    # 1. Convert to lowercase
    text = text.lower()

    # 2. Remove punctuation
    text = text.translate(str.maketrans('', '', string.punctuation))

    # 3. Tokenize
    tokens = word_tokenize(text)

    # 4. Remove stopwords & 5. Lemmatize
    tokens = [
        lemmatizer.lemmatize(word)
        for word in tokens
        if word not in stop_words and word.isalpha()
    ]

    return ' '.join(tokens)

# Apply preprocessing to all reviews
df['cleaned_review'] = df['review'].apply(preprocess_text)

# Show before vs after
print("— Before vs After Preprocessing —\n")
for i in range(3):
    print(f"Original : {df['review'].iloc[i]}")
    print(f"Cleaned   : {df['cleaned_review'].iloc[i]}")
    print("-" * 60)

```

— Before vs After Preprocessing —

```

Original : This product is amazing and works perfectly!
Cleaned   : product amazing work perfectly
-----
Original : Absolutely love it, best purchase I have ever made.
Cleaned   : absolutely love best purchase ever made
-----
Original : Great quality and fast delivery. Highly recommend!
Cleaned   : great quality fast delivery highly recommend
-----

```

✓ Step 5: Feature Extraction using TF-IDF Vectorizer

Machine learning models work with numbers, not text. We use **TF-IDF** (Term Frequency-Inverse Document Frequency)

to convert our cleaned text into numerical feature vectors.

TF-IDF gives higher weight to words that are important in a document but rare across all documents.

```
# — Split data into features (X) and labels (y) —
X = df['cleaned_review']
y = df['sentiment']

# — Train-Test Split (80% train, 20% test) —
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"Training samples : {len(X_train)}")
print(f"Testing samples  : {len(X_test)}")

# — TF-IDF Vectorization —
tfidf = TfidfVectorizer(max_features=500, ngram_range=(1, 2))

X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf  = tfidf.transform(X_test)

print(f"\nTF-IDF feature matrix shape (train): {X_train_tfidf.shape}")
print(f"TF-IDF feature matrix shape (test) : {X_test_tfidf.shape}")
```

```
Training samples : 48
Testing samples  : 12
```

```
TF-IDF feature matrix shape (train): (48, 324)
TF-IDF feature matrix shape (test) : (12, 324)
```

✓ Step 6: Model Training

We'll train two simple yet effective classifiers:

1. **Logistic Regression** – a linear model great for text classification
2. **Multinomial Naive Bayes** – a probabilistic model commonly used in NLP

We'll compare both and pick the best one.

```
# — Model 1: Logistic Regression —
lr_model = LogisticRegression(max_iter=1000, random_state=42)
lr_model.fit(X_train_tfidf, y_train)
lr_predictions = lr_model.predict(X_test_tfidf)
lr_accuracy = accuracy_score(y_test, lr_predictions)
```

```

print(f"📊 Logistic Regression Accuracy: {lr_accuracy:.2%}")

# — Model 2: Multinomial Naive Bayes —
nb_model = MultinomialNB()
nb_model.fit(X_train_tfidf, y_train)
nb_predictions = nb_model.predict(X_test_tfidf)
nb_accuracy = accuracy_score(y_test, nb_predictions)

print(f"📊 Naive Bayes Accuracy          : {nb_accuracy:.2%}")

# — Select the best model —
if lr_accuracy >= nb_accuracy:
    best_model = lr_model
    best_predictions = lr_predictions
    best_name = "Logistic Regression"
else:
    best_model = nb_model
    best_predictions = nb_predictions
    best_name = "Naive Bayes"

print(f"\n🏆 Best Model: {best_name}")

```

📊 Logistic Regression Accuracy: 50.00%

📊 Naive Bayes Accuracy : 50.00%

🏆 Best Model: Logistic Regression

✓ Step 7: Model Evaluation

Let's evaluate our best model using:

- **Accuracy Score** – percentage of correct predictions
- **Classification Report** – precision, recall, and F1-score per class
- **Confusion Matrix** – shows where the model gets confused

```

# — Classification Report —
print(f"=" * 55)
print(f"  Classification Report ({best_name})")
print(f"=" * 55)
print(classification_report(y_test, best_predictions))

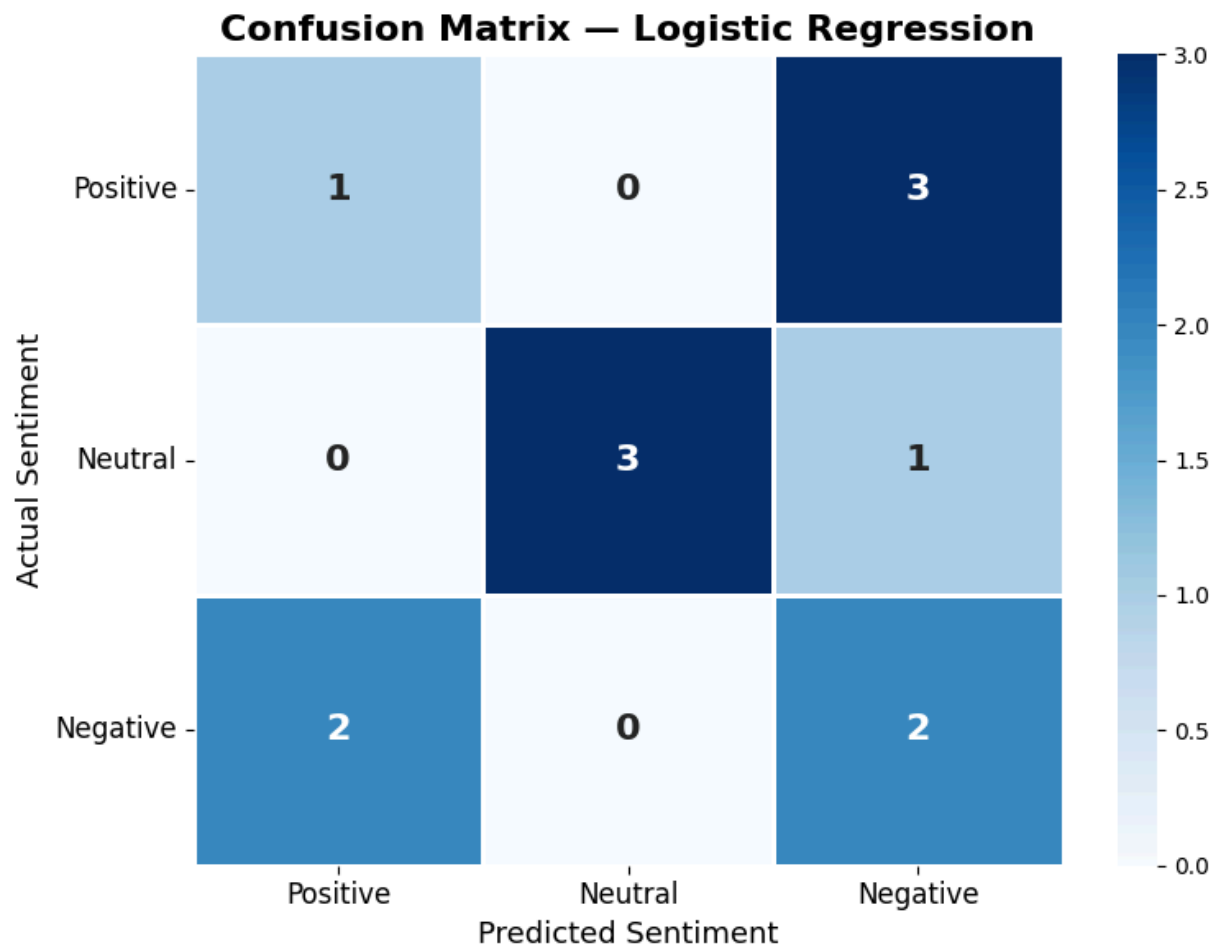
```

Classification Report (Logistic Regression)

	precision	recall	f1-score	support
Negative	0.33	0.50	0.40	4
Neutral	1.00	0.75	0.86	4

Positive	0.33	0.25	0.29	4
accuracy			0.50	12
macro avg	0.56	0.50	0.51	12
weighted avg	0.56	0.50	0.51	12

```
# — Confusion Matrix Visualization —
cm = confusion_matrix(y_test, best_predictions, labels=['Positive', 'Neutral',
plt.figure(figsize=(8, 6))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['Positive', 'Neutral', 'Negative'],
    yticklabels=['Positive', 'Neutral', 'Negative'],
    linewidths=1,
    linecolor='white',
    annot_kws={'size': 16, 'fontweight': 'bold'}
)
plt.title(f'Confusion Matrix - {best_name}', fontsize=16, fontweight='bold')
plt.xlabel('Predicted Sentiment', fontsize=13)
plt.ylabel('Actual Sentiment', fontsize=13)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12, rotation=0)
plt.tight_layout()
plt.show()
```



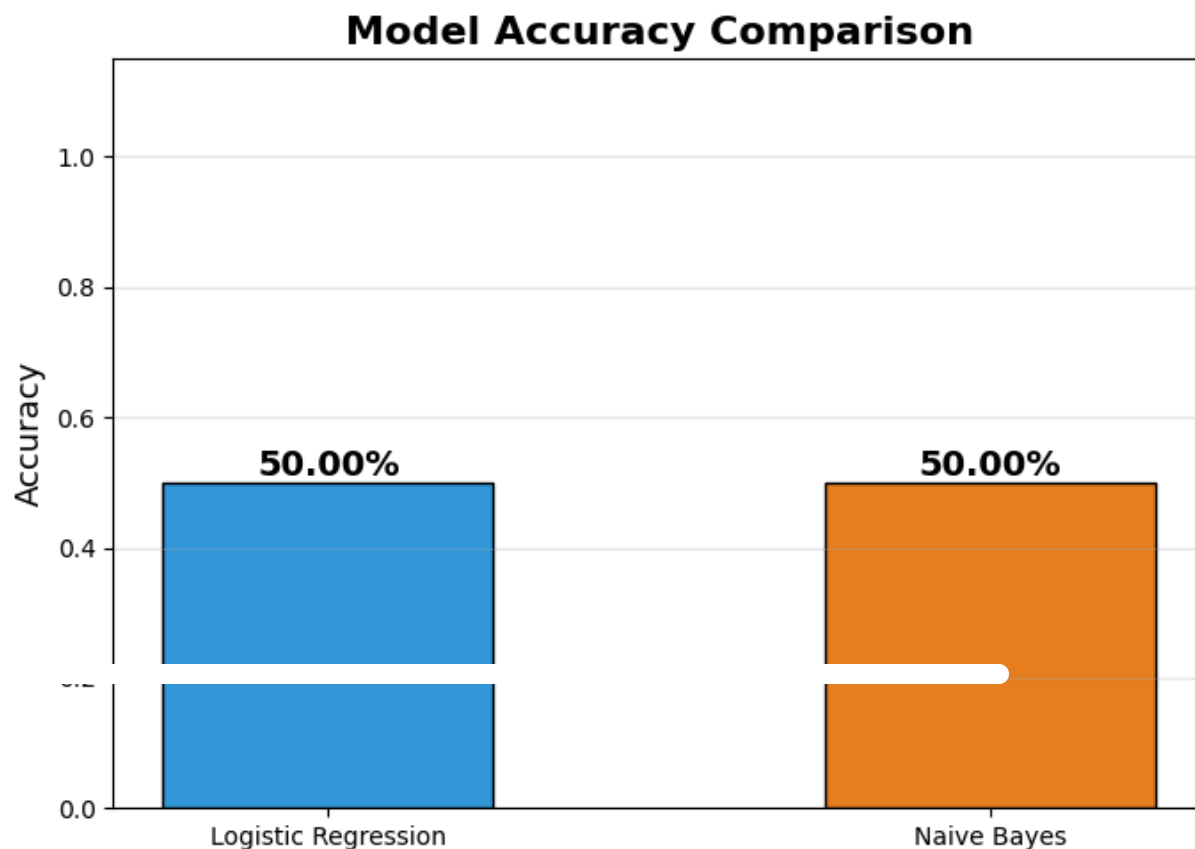
```
# — Model Comparison Bar Chart —
models = ['Logistic Regression', 'Naive Bayes']
accuracies = [lr_accuracy, nb_accuracy]

plt.figure(figsize=(7, 5))
bars = plt.bar(models, accuracies, color=['#3498db', '#e67e22'], edgecolor='black')

for bar, acc in zip(bars, accuracies):
    plt.text(
        bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.01,
        f'{acc:.2%}', ha='center', fontsize=14, fontweight='bold'
    )

plt.title('Model Accuracy Comparison', fontsize=16, fontweight='bold')
plt.ylabel('Accuracy', fontsize=13)
plt.ylim(0, 1.15)
plt.grid(axis='y', alpha=0.3)
```

```
plt.tight_layout()  
plt.show()
```



✓ Step 8: Predict Sentiment for Custom Input

Now let's use our trained model to predict the sentiment of **new customer feedback**. Simply pass any text and the model will classify it as Positive, Negative, or Neutral.

```
def predict_sentiment(text):  
    """Predict sentiment of a given text using the best trained model."""  
    # Preprocess  
    cleaned = preprocess_text(text)  
    # Vectorize  
    vectorized = tfidf.transform([cleaned])  
    # Predict  
    prediction = best_model.predict(vectorized)[0]  
    # Get confidence scores  
    probabilities = best_model.predict_proba(vectorized)[0]  
    classes = best_model.classes_  
  
    print(f"📄 Input      : {text}")
```

```

print(f"🔍 Cleaned : {cleaned}")
print(f"🎯 Sentiment : {prediction}")
print(f"📊 Confidence:")
for cls, prob in zip(classes, probabilities):
    bar = '█' * int(prob * 30)
    print(f"    {cls:10s} : {prob:.2%} {bar}")
print("-" * 55)
return prediction

# — Test with sample inputs —
print("=" * 55)
print("  Custom Sentiment Predictions")
print("=" * 55 + "\n")

test_reviews = [
    "This is the best thing I have ever bought!",
    "Terrible quality, broke after two days.",
    "It is okay, works as expected.",
    "Absolutely fantastic! Exceeded all my expectations.",
    "Very poor packaging, item was damaged."
]

for review in test_reviews:
    predict_sentiment(review)
    print()

```

Custom Sentiment Predictions

📄 Input : This is the best thing I have ever bought!

🔍 Cleaned : best thing ever bought

🎯 Sentiment : Positive

📊 Confidence:

Negative	: 34.36%	██████████
Neutral	: 25.88%	██████
Positive	: 39.76%	██████████

📄 Input : Terrible quality, broke after two days.

🔍 Cleaned : terrible quality broke two day

🎯 Sentiment : Negative

📊 Confidence:

Negative	: 43.15%	██████████
Neutral	: 26.57%	██████
Positive	: 30.27%	██████

📄 Input : It is okay, works as expected.

🔍 Cleaned : okay work expected

🎯 Sentiment : Neutral

📊 Confidence:

Negative	: 31.63%	██████
----------	----------	--------

```
Neutral : 37.49% ██████████
Positive : 30.88% ██████████
```

```
📄 Input      : Absolutely fantastic! Exceeded all my expectations.
🔍 Cleaned    : absolutely fantastic exceeded expectation
🎯 Sentiment  : Positive
📊 Confidence:
Negative     : 26.27% ████████
Neutral      : 28.70% ████████
Positive     : 45.03% ██████████
```

```
📄 Input      : Very poor packaging, item was damaged.
🔍 Cleaned    : poor packaging item damaged
🎯 Sentiment  : Negative
📊 Confidence:
Negative     : 39.44% ██████████
Neutral      : 27.04% ████████
Positive     : 33.52% ██████████
```

✎ Try Your Own Review!

Change the text in the cell below to test any feedback you'd like.

```
# — 🗨️ Type your own review here! —
my_review = "The product is really good and I love the design!"

predict_sentiment(my_review)
```

```
📄 Input      : The product is really good and I love the design!
🔍 Cleaned    : product really good love design
🎯 Sentiment  : Positive
📊 Confidence:
Negative     : 25.90% ████████
Neutral      : 30.92% ████████
Positive     : 43.17% ██████████
```

```
'Positive'
```

✎ Step 9: Top Positive & Negative Words ⚡ (Bonus)

Let's find out which words the model considers most strongly associated with **Positive** and **Negative** sentiments. This uses the Logistic Regression model coefficients to rank feature importance.

```
# — Top words per sentiment (only works with Logistic Regression) —
feature_names = np.array(tfidf.get_feature_names_out())

# Get class indices
classes = lr_model.classes_
print(f"Classes: {classes}\n")

fig, axes = plt.subplots(1, 3, figsize=(18, 6))
colors_map = {'Positive': '#2ecc71', 'Negative': '#e74c3c', 'Neutral': '#3498db'}

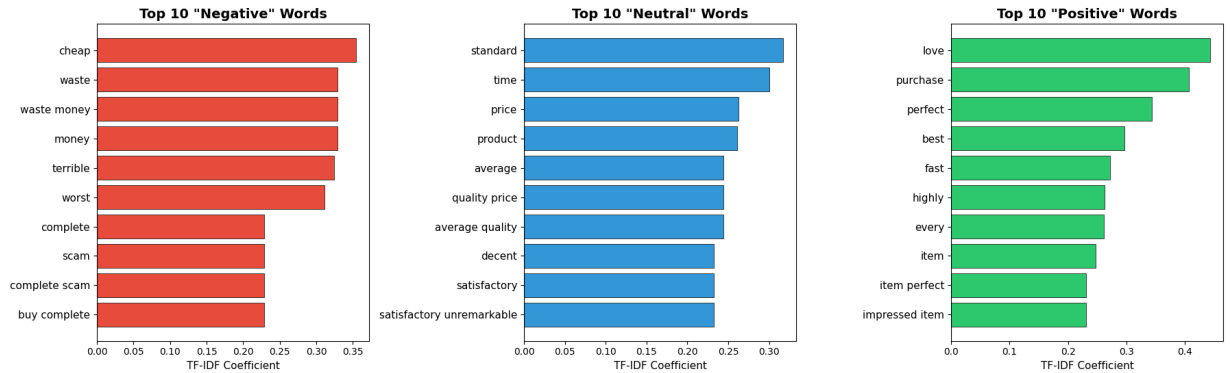
for idx, (cls, ax) in enumerate(zip(classes, axes)):
    # Get coefficients for this class
    coefs = lr_model.coef_[idx]
    top_indices = np.argsort(coefs)[-10:] # Top 10
    top_words = feature_names[top_indices]
    top_scores = coefs[top_indices]

    ax.barh(top_words, top_scores, color=colors_map.get(cls, '#95a5a6'), edgecolor='black')
    ax.set_title(f'Top 10 "{cls}" Words', fontsize=14, fontweight='bold')
    ax.set_xlabel('TF-IDF Coefficient', fontsize=11)
    ax.tick_params(axis='y', labelsize=11)

plt.suptitle('Most Important Words per Sentiment Class', fontsize=17, fontweight='bold')
plt.tight_layout()
plt.show()
```


Classes: ['Negative' 'Neutral' 'Positive']

Most Important Words per Sentiment Class



✓ Step 10: Word Cloud Visualization ⚡ (Bonus)

A **word cloud** gives a visual representation of the most common words in each sentiment category.

Larger words appear more frequently in the reviews.

```
# Install wordcloud if not already installed
try:
    from wordcloud import WordCloud
    print("✅ wordcloud is already installed.")
except ImportError:
    print("⌚ Installing wordcloud...")
    import subprocess, sys
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', 'wordcloud'],
                          from wordcloud import WordCloud
    print("✅ wordcloud installed successfully!")
```

✅ wordcloud is already installed.

Word Clouds by Sentiment



Step 11: Review Length Analysis

Let's see if there's any relationship between the **length of a review** and its **sentiment**. This can reveal interesting patterns — for example, negative reviews might tend to be longer

because customers explain their complaints in detail

```
# — Add word count column —
df['word_count'] = df['cleaned_review'].apply(lambda x: len(x.split()))

plt.figure(figsize=(9, 5))
sns.boxplot(
    data=df,
    x='sentiment',
    y='word_count',
    palette={'Positive': '#2ecc71', 'Negative': '#e74c3c', 'Neutral': '#3498db'},
    order=['Positive', 'Neutral', 'Negative']
)
plt.title('Review Word Count by Sentiment', fontsize=16, fontweight='bold')
plt.xlabel('Sentiment', fontsize=13)
plt.ylabel('Word Count (after cleaning)', fontsize=13)
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```